VOLUME 03 / 08

BUILD THE PIPELINE

Divide instruction work into five valid stages

Plan stage boundaries, carry control with data, implement edge updates and inspect a correct pipeline trace.

FIG 05A / LOGICAL STAGES



Illustration: stage boundaries hold the instruction and its control state.

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

What you will build

You will design the stage register contents and explain how independent instructions overlap before introducing dependency handling.

Before you start

Complete instruction result calculations from Volume 2. Preserve a reference revision.

Files and outputs

rtl/rv32_core.sv: pc, f_*, x_*, m_*, w_* and rising-edge block. tests/tb.sv: retirement output.

Build order

1. Specify the work and outputs of each stage.
2. List every field crossing each boundary.
3. Implement reset and unconditional older-stage advance.
4. Issue spaced independent programs.
5. Add retirement records and draw a cycle table.

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

One instruction moves through a datapath

A **datapath** carries values through hardware. A control decoder selects the operation, input muxes and permitted side effects. The **ALU**, or arithmetic logic unit, computes integer arithmetic and bitwise results.

For `add x3, x1, x2`, fetch obtains the instruction, decode identifies the registers, the register file supplies operands, and the ALU adds them. A write-enable permits the result to update x3.

A load instead uses the ALU to add the base register and immediate offset. Data memory supplies the final value. A store uses the calculated address plus a separate store-data operand.

A single-cycle implementation completes this path within one clock period. A long path through instruction memory, decode, registers, ALU and data memory can force a long clock.

Control and data must refer to the same instruction. If a pipeline carries a result but loses its destination or write-enable, the value can update the wrong state.

FIG 04 / ADD DATAFLOW

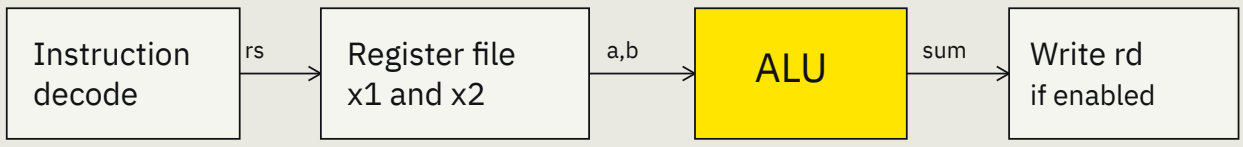


Illustration: values move forward while control selects the permitted update.

Different instructions share the machine over time

The five stages are instruction fetch (IF), instruction decode (ID), execute (EX), memory access (MEM) and writeback (WB). Each stage performs different work on a different instruction during the same clock interval.

Registers between stages preserve each instruction's state. A **valid bit** says whether a stage contains an actual instruction. An invalid entry is a **bubble**. A bubble must never update architectural registers or memory.

In the RTL, `f_*` is the IF/ID register and feeds ID. `x_*` feeds EX, `m_*` feeds MEM and `w_*` feeds WB. The name prefix describes stored state, not an additional stage. The fetch PC can be ahead of WB. Each instruction carries its own saved PC.

> **IDEAL TIMING**
>
> For N independent instructions, an ideal five-stage pipeline completes the last WB in `N + 4` cycles, counting initial fill and no stalls. For large N, throughput approaches one retired instruction per cycle.

One instruction still crosses all five stages. Pipeline throughput and instruction latency answer different questions.

FIG 05A / LOGICAL STAGES



Illustration: stage boundaries hold the instruction and its control state.

Step 1: assign work to stages

IF presents PC to instruction memory. ID reads registers and captures the instruction's context. EX performs the ALU operation, target decision or address calculation. MEM performs the data-port transaction. WB updates the register file and reports completion.

Draw five columns and place every signal-producing operation in one column. Mark the clock boundary between columns. Keep the assumed combinational memory read visible; moving to synchronous RAM changes this allocation.

The reference names stage storage by prefixes: f is IF/ID, x is ID/EX, m is EX/MEM and w is MEM/WB. There is no extra “f stage” beyond the five logical stages.

For an ADD, MEM still carries its result onward even though no data read occurs. For a store, WB produces a trace record even though no register destination exists.

Keep instruction identity attached to its work. Saved PC and instruction bits make debugging possible and prevent using the speculative fetch PC for older instructions.

Step 2: list each pipeline payload

Write the boundary contents before coding. Start with ADD, then extend the list for loads, stores and control flow.

IF/ID	Valid bit, fetched PC and instruction word.
ID/EX	Valid bit, PC, instruction and the two captured register values.
EX/MEM	Valid bit, PC, instruction, result/address, store data, destination, write/load/store controls, trap and funct3.
MEM/WB	Valid bit, PC, instruction, final result, destination, write control and trap.

The reference keeps instruction bits through EX rather than storing a fully expanded decoded control bundle in ID/EX. That is one legitimate organization, not a universal requirement.

Trace a store's source value separately from its address. Trace a load's size/signedness until MEM. Trace a jump's link until WB.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 3A. Which boundary must retain a load’s funct3, and why can it not disappear after EX?

Step 3: implement the older-stage updates

Start with the steady flow at the back of the pipeline. These updates occur on every ordinary non-reset edge, including a front-end stall.

```
w_valid ⩽ m_valid;
w_pc ⩽ m_pc; w_insn ⩽ m_insn; w_rd ⩽ m_rd;
w_write ⩽ m_write; w_trap ⩽ m_trap;
w_result ⩽ m_load ? load_value : m_result;
m_valid ⩽ x_valid;
m_pc ⩽ x_pc; m_insn ⩽ x_insn;
m_result ⩽ result; m_store ⩽ b;
```

This excerpt omits other EX/MEM fields for space. Read and implement the complete destination/control assignments in the actual source. All right-hand sides refer to pre-edge values.

If a load stalls the younger consumer, freezing these older stages too would stop the producer from reaching WB. A front-end stall must create space while allowing the required value to advance.

For the first dependency-free experiment, omit special stall behavior only in an explicitly labeled learning version and use NOP spacing. The finished core must implement Volume 4 before ordinary dependent programs are correct.

Step 4: implement valid bits and reset

A valid bit is the authority for side effects. An invalid stage may still contain stale register numbers or instruction bits. Do not interpret those bits as a live operation.

```
if (reset) begin
    pc ⩽ 0;
    f_valid ⩽ 0; x_valid ⩽ 0;
    m_valid ⩽ 0; w_valid ⩽ 0;
end
```

The reference also initializes stored data/control fields and clears the register file. Reset is synchronous and active high. External memories are not reset through the core.

Every memory write must require a valid MEM instruction. Every register write must require a valid, non-trapping WB writer and rd≠0. Keep the x0 rule even if a test never writes x0.

To insert a bubble, clear the destination valid bit. Encoding an ADDI x0,x0,0 is not the same representation: it remains a real valid instruction and can alter retirement counts.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 3B. Why is x_insn=0 not a safe way to represent a bubble in this core?

Step 5: capture the front end

When nothing holds or redirects the front end, transfer the decoded instruction context into ID/EX, capture the new fetch in IF/ID and advance PC by four.

```
x_valid ⩽ f_valid; x_pc ⩽ f_pc; x_insn ⩽ f_insn;
f_valid ⩽ !halted; f_pc ⩽ pc; f_insn ⩽ imem_rdata;
if (!halted) pc ⩽ pc + 4;
```

The complete ID operand capture needs the WB-to-ID bypass explained in Volume 4. A plain read from regs can be stale at a coincident WB edge.

Temporarily spaced programs help isolate stage-transfer errors. Use `pipeline_spaced` in the teaching labs. Check PC sequence and instruction identity, not just the final sum.

If PC values jump or repeat without an intended redirect, inspect whether a stage saved the old PC or incorrectly read the current fetch PC. Nonblocking assignments make the old/new distinction precise.

Step 6: create an observable retirement record

The local interface reports one completed WB-stage instruction at a time. It carries valid, PC, instruction, destination, value and trap, plus an interrupt flag that only the ZICSR=1 build sets. riscv-formal reads a separate RVFI trace.

```
retire_valid ⩽ w_valid;
retire_pc ⩽ w_pc; retire_insn ⩽ w_insn;
retire_trap ⩽ w_valid && w_trap;
if (w_valid && w_write && !w_trap && w_rd ≠ 0)
    regs[w_rd] ⩽ w_result;
```

For nonwriters, report no architectural register destination/value. For x0, report no retained result. Keep trap records distinguishable from ordinary completion.

The testbench prints STORE events when memory changes and RET records after the WB update has settled. Store memory effects occur in MEM, before their local WB notification.

Run a three-instruction independent program, write its IF/ID/EX/MEM/WB table, and compare it with the saved PCs in the printed trace. Do not infer stage timing only from text-output order.

Step 7: check fill, drain and lost instructions

For N independent instructions entering an empty five-stage pipeline and draining normally, the ideal stage-interval count is N+4. Testbench reset and halt bookkeeping can use a different counting window, so state your convention.

Count each input instruction once in the retirement stream. Check that no initial garbage instruction retires after reset. Check that the last ordinary instruction is not lost when EBREAK follows it.

Test a sequence with unique PCs and unique results. Repeated NOPs are useful spacing but poor identifiers for diagnosing a duplicate or missing instruction.

Run `make test lint` after implementing the complete pipeline and hazard handling. Keep the intermediate spaced result as a teaching artifact, not the final correctness claim.

Debugging order

Find the first incorrect boundary, check its valid bit, then compare its saved instruction/PC/control with the previous stage. Repair the earliest divergence rather than compensating downstream.

A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

Question	Does stage overlap preserve instruction identity?
Prediction	Each valid input instruction produces one ordered local completion record.
Experiment	Trace a spaced sequence with unique PCs/results through all boundaries.
Evidence	Cycle table, boundary observations and retirement list.

What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

Answers and completion check

3A. EX/MEM must preserve the size/signedness field until MEM formats the loaded word. An address alone cannot distinguish LB from LBU or LW.

3B. The zero instruction word is unsupported and traps if treated as valid. A bubble is a false valid bit, independent of stale instruction contents.

Completion check

You can list all boundary payloads, explain old/new edge state, trace fill/drain and show one retirement per valid instruction. Move to hazards before running dependent code without spacing.

Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. Brown pipeline notes
2. Project source
3. RISC-V ISA

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

Keep the evidence boundary

Teaching labs are separate from the 253-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, ISA certification or formal proof is implied. The DOT4 speedup covers one kernel.

Continue with Volume 4. Run the complete regression before treating a learning edit as a finished implementation.